

Payroll Run: Calculation Guide & Review

Source: `backend/app/routes/payroll_run/calculations/`, plus the helpers in `payroll_runs_widgets/helpers_functions.py`, `sections/*.py` and `employees.calculate_number_of_days`. Generated 5 Oct 2026.

1. Shared terms used by every calculation

Term	What it means in the code
Period days	Calendar days in the payroll period, counting both the first and last day: <code>(end - start) + 1</code> . June = 30, July = 31, Feb = 28/29.
Based value (“salary” below)	The amount a calculation is based on. It is the sum of the employee's <i>based elements</i> for that payroll element (for example Basic + Housing), each one added or subtracted according to its <code>type</code> . Values come from the employee master screen (<code>employees_payrolls</code>), not from earlier results in the same run. Built in <code>sections/data.py</code> → <code>employee_element_value</code> .
Leave days (<code>l_days</code>)	The part of the leave that falls inside this period. If the leave type is “ <i>Calendar Days</i> ”, every day counts. Otherwise only working days count, which excludes the legislation's weekend days and public holidays.
Legislation	The employee's legislation document. It holds the leave limits, overtime hours, tax rates, social security ceilings and gratuity days.
Rounding	Every result is rounded to 2 decimals.
Re-runs	If an element was already processed in the same period, it is skipped. On save, its value is also forced to 0 if the same employee + element already exists in that period.

All examples below use: **Basic salary = 10,000**, **period = June 2026 (30 days)**, unless stated otherwise.

2. Regular payroll elements

2.1 Input Value (pro-rated fixed amount) EARNING / ANY

input_value.py · py_input_value_ff

Pays a fixed monthly amount (salary, allowance, and so on), reduced when the employee or the element covers only part of the month.

From = latest of (hire date, element start, period start)
To = earliest of (end of service, element end, period end)
Days worked = (To - From) + 1 (never below 0)

Result = Element value × Days worked ÷ Period days

Example: Joined on 11 June → days worked = 20.

$10,000 \times 20 \div 30 = 6,666.67$

Uses **actual calendar days** of the month, so the daily rate changes from month to month (10,000/28 in Feb vs 10,000/31 in July). Leave days taken are not subtracted here; leave is handled by separate elements.

✓ **Formula is correct for calendar-day pro-rating.**

2.2 Annual Leave Entitlement (monthly accrual) ACCRUAL

annual_leave_entitlement.py · py_annual_leave_entitlement_ff

Works out how many leave days the employee earned this month and what they are worth.

Days worked = same as Input Value
Days earned = Yearly entitlement ÷ 12 × Days worked ÷ Period days
Value = Days earned ÷ 30 × Based salary

Example: Entitlement 30 days/year, full month.

Days earned = $30 \div 12 \times 30/30 = 2.5$ days.

Value = $2.5 \div 30 \times 10,000 = 833.33$ (one-twelfth of a month's salary)

The daily rate here is **salary ÷ 30**. Taking annual leave (2.3) uses **salary × 12 ÷ 365**, so a day earned and a day taken are not valued the same way.

✓ **Formula is correct.**

2.3 Overtime – Normal and Holidays EARNING

overtime_normal.py · overtime_holidays.py

The two functions are identical. The only difference is which “working hours” field they read from the legislation.

Hourly rate = Based salary ÷ Period days ÷ Daily working hours

Result = Overtime hours (element value) × Hourly rate

Example: 10 hours, 8 working hours/day.

$10 \div 8 \div 30 \times 10,000 = 416.67$ (the same 10 hours in February = 446.43)

No overtime premium is applied. The hours are paid at 1.0× the hourly rate. If the law needs 1.25× (normal) or 1.5× (holiday/night), the multiplier is missing, and normal and holiday overtime pay exactly the same. The only workaround is to enter weighted hours.

If `number_of_working_hours_for_overtime_*` is missing from the legislation, the default is **0**, which causes a **division-by-zero crash**.

The hourly rate depends on month length (÷28 vs ÷31). Many policies use a fixed 30.

2.4 Non-recurring element (bonus, one-off) ANY

nonrecurring.py · py_nonrecurring_ff

If the element's dates overlap the period → Result = full element value (no pro-rating)

Otherwise → nothing

Example: Bonus 2,000 dated 15 June → **2,000**.

If a non-recurring element has **no end date** (or an end date in a later month), it overlaps **every** later period and is paid **every month**. Make sure the end date always falls inside the same period, or enforce this in the code.

2.5 Service Tax DEDUCTION

service_tax.py · py_service_tax_ff

Result = Based value × Service tax % ÷ 100

Example: 1% of 10,000 = **100.00**

✓ **Correct.** (Not pro-rated for partial months, so it is based on the full master value.)

2.6 Income Tax (progressive) DEDUCTION

income_tax_deduction.py · helpers: calculate_progressive_income_tax, income_tax_brackets

```
Annual taxable = max(Monthly based value - Monthly exemption, 0) × 12
Annual tax      = Σ over brackets: (part of income inside the bracket) × bracket
%
Monthly tax    = Annual tax ÷ 12
```

If no brackets are defined: one bracket from 0 to income_tax_ceiling (or unlimited) at income_tax_percentage.

Example: Monthly taxable 10,000, exemption 1,500/month → annual 102,000.

Brackets: 0–40,000 @0%, 40,000–55,000 @10%, 55,000–70,000 @15%, 70,000+ @20%

= 15,000×10% + 15,000×15% + 32,000×20% = 1,500 + 2,250 + 6,400 = 10,150 per year → **845.83** per month

- The element value is treated as a **monthly** exemption. If you store an annual exemption (such as 20,000/year), it will be wrong by ×12.
- Tax uses the **master** values only. Overtime, bonuses, unpaid-leave deductions and the *calculated* social security in the same run are **not** included unless they are set up as based elements with master values.
- Brackets must touch exactly (0–40,000 then 40,000–55,000). Writing “40,001” leaves 1 unit untaxed. A bracket with an empty “to” in the middle taxes everything above it.

2.7 Social Security – Employee share and Employer share EMPLOYEE EMPLOYER

social_security_employee.py · social_security_employer.py

```
Insurable salary = sum of based elements, but for each one it takes the record
that
                                ends EARLIEST within the current year (open-ended records are
used only
```

```
                                if none end this year)
Rule row                = the ceiling row whose start ≤ period end and (end empty or ≥
period end)
```

```
                                (if several match, the LAST one wins)
Ceiling                = rule row ceiling (if 0/empty → no ceiling)
```

```
Employee share = Employee % × min(Insurable salary, Ceiling)
```

```
Employer share = Employer % × min(Insurable salary, Ceiling)
```

Example: Insurable 10,000, ceiling 8,000, employee 11%, employer 18.75%.

Employee = 11% × 8,000 = **880.00** Employer = 18.75% × 8,000 = **1,500.00**

- **Confirm the “earliest-ending record this year” rule is what you want.** Example: salary 8,000 until 31 Mar, then 10,000 open-ended. Every month of the year (including December) uses **8,000**. That matches a “January salary fixed for the year” rule, but it is wrong if contributions should follow the current salary.
- No **minimum** insurable salary is applied, and there is **no pro-rating** for joiners or leavers in their first and last month.
- If no ceiling row matches the date, the percentage is 0 and the result is 0 (no warning). If `social_security_ceilings` is missing, it crashes.
- If the element has no based elements, this raises a “no value found” error. It does not fall back to the element's own value.

2.8 End-of-Service Gratuity Accrual PROVISION

gratuity_accrual.py · helper get_previous_gratuity_accrual

```
Service days      = (earliest of end of service / element end / period end - hire date) + 1
```

```
First 5 yrs days = min(Service days, 1,825)
```

```
After 5 yrs days = max(Service days - 1,825, 0)
```

```
Gratuity days     = First5 ÷ 365 × 21 + After5 ÷ 365 × 30      (21 / 30 come from legislation)
```

```
Total liability   = Gratuity days × Basic salary ÷ 30
```

```
This month        = Total liability - Sum of all gratuity already booked ("Gratuity Balance")
```

Example: Hired 1 Jul 2019, basic 10,000, run for June 2026.

Service days = 2,557 → 1,825 + 732

Gratuity days = 1,825/365×21 + 732/365×30 = 105.00 + 60.16 = 165.16

Liability = 165.16 × 10,000 ÷ 30 = 55,054.79

Booked to end of May = 54,232.88 → **June accrual = 821.92**

✓ The “true-up” method is correct. A salary increase revalues the whole past liability in the month it happens (a decrease gives a negative accrual).

- No **cap** (UAE law caps total gratuity at 2 years' wage).
- Unpaid leave days are not removed from service (UAE law excludes them).
- Uses 365 days per year (leap days ignored). This is minor.
- The “Gratuity Balance” lookup has no `company_id` filter. If two companies share the database and both have a balance with this name, their configurations are mixed.

3. Leave calculations

Every leave calculation except Annual Leave returns the **amount to deduct** (the unpaid part). The linked payroll element must be set up as a **deduction**.

3.1 Annual Leave (leave taken) **EARNING**

annual_leave.py · py_annual_leave_ff

Leave days = working days of the leave inside this period
Result = Based salary × Leave days × 12 ÷ 365

Example: 5 working days → $10,000 \times 5 \times 12 \div 365 = 1,643.84$

- `is_pay_in_advanced` is passed in but **never used**. A leave that runs into next month is always paid month by month.
- This produces a **positive** amount while the full salary is still paid through Input Value. Confirm this is meant to be extra “leave salary”. If it is not, it doubles pay for those days.
- The daily rate ($\times 12/365$) differs from accrual ($\div 30$) and from unpaid leave ($\div$ period days).

3.2 Unpaid Leave **DEDUCTION**

unpaid_leave.py · py_unpaid_leave_ff

Result = Based salary × Leave days ÷ Period days

Example: 5 days → $10,000 \times 5 \div 30 = 1,666.67$

Basis mismatch: if the leave type counts **working days**, the top of the fraction is working days but the bottom is **calendar** days. A whole month of unpaid leave in a 5-day week gives about $22 \div 30 = 73\%$ deducted, although the employee did not work at all. Set unpaid leave types to “*Calendar Days*”, or divide by the working days in the period.

3.3 Sick Leave (full → half → unpaid) DEDUCTION

sick_leave.py · helper get_used_leave_days

Used before = calendar days of earlier Posted "SL" leaves in the same calendar year

+ days of THIS leave that fell in earlier periods

Full-pay days left = $\max(0, \text{Full limit} - \text{Used before})$

Full-pay days = $\min(\text{Leave days}, \text{Full-pay days left})$ → deduct 0%

Half-pay days left = $\max(0, \text{Half limit} - \max(0, \text{Used before} - \text{Full limit}))$

Half-pay days = $\min(\text{remaining}, \text{Half-pay days left})$ → deduct 50%

Unpaid days = whatever is left → deduct 100%

Result = $\text{Salary} \div \text{Period days} \times (\text{Half-pay days} \times 0.5 + \text{Unpaid days})$

Example A: limits 15 full / 30 half, 10 days used before, 10 days now.

Full left 5 → 5 full; 5 half → $10,000 \div 30 \times 5 \times 0.5 = \mathbf{833.33}$

Example B: 50 used before, 10 now → full left 0, half left 30 – 35 → 0 → 10 unpaid = **3,333.33**

✓ The tier logic (full → half → unpaid) is correct.

- **Used-before days are counted in calendar days**, but current leave days can be **working days**. The full and half tiers are then used up faster than they should be.
- `get_used_leave_days` finds the leave type by `code = "SL"` with **no company filter**. It may pick another company's SL type, which gives 0 used days and too much full pay. It crashes if no "SL" type exists.
- The year is the calendar year of the leave start, not the service year.
- Same working-days ÷ calendar-days mismatch as Unpaid Leave.

3.4 Maternity, Paternity and Compassionate Leave DEDUCTION

maternity_leave.py · paternity_leave.py · compassionate_leave.py (identical logic)

Used before = days of THIS leave that fell in earlier periods (calendar days)

Full-pay left = $\max(0, \text{Paid-days limit} - \text{Used before})$

Full-pay days = $\min(\text{Leave days}, \text{Full-pay left})$ → deduct 0%

Rest = $\text{Leave days} - \text{Full-pay days}$ → deduct 100%

Result = $\text{Salary} \times \text{Rest} \div \text{Period days}$

Example (maternity): limit 45, leave 10 May → 31 Jul (calendar-day type).

June: used before = 22, leave days = 30, full left = 23 → 23 paid, 7 deducted

$10,000 \times 7 \div 30 = \mathbf{2,333.33}$

- **No half-pay tier.** UAE maternity is 45 days full pay + 15 days half pay. Here everything after the paid limit is deducted at 100%.
- Only the same leave is counted as "used". Two separate compassionate or paternity leaves in a year each get the full limit again.
- If the limit field is missing from the legislation, it defaults to 0, so the whole leave is deducted.
- Same calendar vs working-day mismatch as Sick Leave.

4. Loans & Advances

4.1 Loan / Advance installment **DEDUCTION**

loan_and_advances.py · sections/loans.py

```
Paid to date = sum of all values already booked for this loan in payroll runs  
Remaining   = max(Total amount - Paid to date, 0)  
Result      = min(Monthly installment, Remaining)      (0 → skipped)
```

Example: Loan 12,000, installment 1,000, already paid 11,500 → $\min(1,000, 500) = 500.00$ (last installment)

✓ **Correct. The deduction starts once the deduction date $\leq$ period end.**

5. Issues found (priority list)

Level	Issue	Where / why it matters
HIGH	Based salary is doubled when a salary changes mid-month	<code>sections/data.py</code> <code>employee_values_by_name</code> adds up every record that overlaps the period. Example: old basic ends 15 June, new basic starts 16 June. The based value becomes old + new (about 2× salary), and that flows into leave deductions, overtime, accruals, income tax and gratuity. (Input Value is fine because it pro-rates each record separately.) The fallback <code>get_employee_element_value</code> in <code>helpers</code> has the same issue.
HIGH	Wrong employee's value can be used	<code>sections/data.py</code> <code>employee_payrolls_by_id</code> is keyed by element definition only (the comment says “this was ['_id']”). It is not keyed per employee. If an element has no based elements , <code>employee_element_value</code> returns the value of whichever employee was loaded last.
HIGH	Overtime has no premium multiplier	Normal and holiday overtime both pay 1.0× the hourly rate.
HIGH	Working days ÷ calendar days in leave deductions	Unpaid, sick, maternity, paternity and compassionate leave under-deduct when the leave type counts working days.
MED	Non-recurring paid every month if end date is open	<code>nonrecurring.py</code> plus the open end date being treated as <code>datetime.max</code> .
MED	Sick leave “used days” counts calendar days, and the “SL” lookup has no company filter	<code>helpers_functions.get_used_leave_days</code>
MED	No half-pay tier for maternity	Needed if the legislation has full + half pay (UAE: 45 + 15).
MED	Social security uses the earliest-ending salary record of the year	Confirm this is intended. There is also no minimum wage floor and no pro-rating.
MED	Income tax ignores in-run items (overtime, bonuses, unpaid leave, calculated social security)	Only master values are taxed.
MED	Gratuity: no 2-year cap, unpaid leave not excluded	<code>gratuity_accrual.py</code>
LOW	Three different daily-rate bases	÷30 (accrual, gratuity), ×12/365 (annual leave), ÷period days (everything else).
LOW	<code>is_pay_in_advanced</code> unused	<code>annual_leave.py</code>
LOW	Crashes on missing configuration	Overtime hours = 0 (divide by zero). <code>social_security_ceilings</code> missing. <code>sections/leaves.py</code> calls <code>function.upper()</code> when the based element has no function (only the first branch checks for None).
LOW	Misleading error text	<code>sections/leaves.py</code> says “missing number_of_days for annual leave calculation” for every leave type. A leftover <code>print()</code> is in <code>social_security_employer.py</code> .

6. How to check a payslip by hand

1. Write down the period days (28–31) and the employee's based value for each element (check its based elements and whether each one adds or subtracts).
2. Earnings: $\text{Input Value} = \text{value} \times \text{days worked} \div \text{period days}$. Overtime = $\text{hours} \div \text{daily hours} \div \text{period days} \times \text{salary}$.
3. Leaves: count the leave days inside the month (working or calendar, depending on the leave type). Apply the tiers (sick: full $\rightarrow$ half $\rightarrow$ unpaid; maternity, paternity, compassionate: paid limit $\rightarrow$ unpaid). Deduction = $\text{salary} \times \text{days} \div \text{period days} (\times 0.5 \text{ for half pay})$.
4. Deductions: social security = $\% \times \min(\text{salary}, \text{ceiling})$. Income tax = brackets on (monthly – exemption) $\times 12$, then $\div 12$. Loan = $\min(\text{installment}, \text{remaining})$.
5. Employer side: employer social security, and gratuity accrual = new total liability – amount booked so far.
6. Compare each figure with the run line by line (to 2 decimals).